Fulminate

Testing CN Separation-Logic Specifications in C

Rini Banerjee

University of Cambridge

Code Reasoning Spotlight series
Amazon

7 May 2025

Why test specifications?

#1 Fits **test-debug cycle** engineers are familiar with

More familiarity => more likely to be adopted

#2 Helps to guide the specification-writing process

Specifications can be checked at runtime 'as you go', flagging mistakes quickly

#3 **On-ramp** to full formal verification

Gradual assurance, rather than "all-or-nothing"

#4 Both testing **and** proof from the same specifications

2-in-1: Annotations only need to be written once

Testing specifications in C

```c
signed int increment(signed int i)
/*@ requires i < power(2, 31) - 1 @*/
/*@ ensures return == i + 1 @*/
{
  return i + 1;
}
```

Source-to-source translation

```c
signed int increment(signed int i)
{
  check_increment_precondition(i);
  signed int ret_value = i + 1;
  check_increment_postcondition(i, ret_value);
  return ret_value;
}
```

Prior work: E-ACSL

- Executable fragment of ACSL, Frama-C's specification language
- Uses **first-order logic**
  - Hard to capture complex C ownership properties

Common Specification Language for Static and Dynamic Analysis of C Programs,
Mickaël Delahaye, Nikolai Kosmatov, Julien Signoles. SAC 2013.

Executing separation logic specifications

- Existential quantification over $*$ blows up quickly
  - $2^{\texttt{size}(h)}$ possible heap splittings for a heap h
  - General case does not lend itself well to runtime testing

$$\exists p, q \,.\, p \mapsto v * q \mapsto w$$

Fulminate

Testing CN **Separation-Logic** Specifications in C

✔ Captures complex ownership patterns of imperative programs

CN specification language

Intentionally *inexpressive*

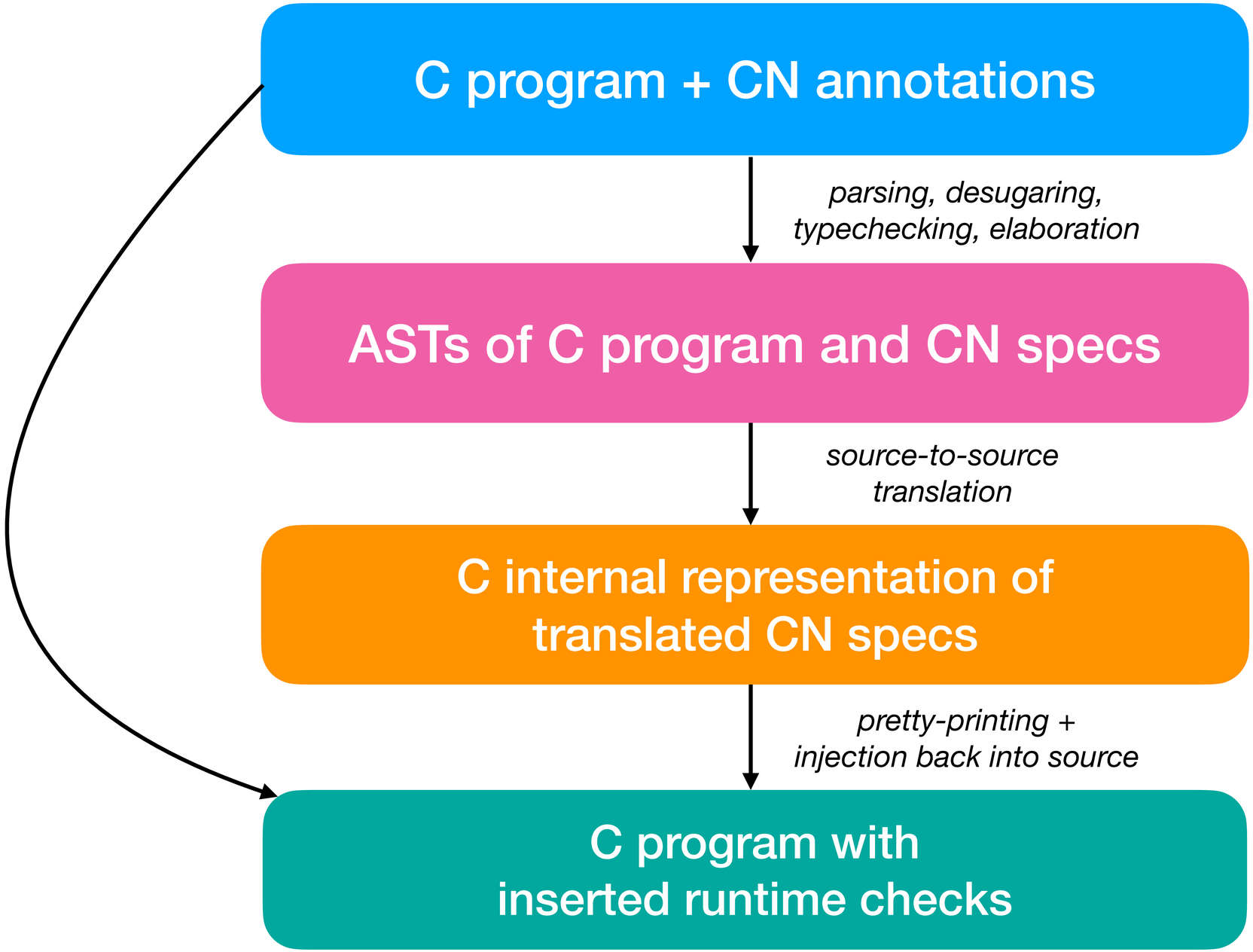
Input/output variables rather than unbounded quantification

$\Longrightarrow$

✔ Good for SMT solver

✔ Makes executability of specifications computationally feasible

$$\exists v \,.\, p \mapsto v \qquad\qquad \textbf{take}\ v = \texttt{Owned}(p)$$


C program + CN annotations
parsing, desugaring,
typechecking, elaboration
ASTs of C program and CN specs
source-to-source
translation
C internal representation of
translated CN specs
pretty-printing +
injection back into source
C program with
inserted runtime checks

Linked list example in CN

$$\mathtt{IntList}(p, xs) =$$
$$(p = \mathtt{NULL} \wedge$$
$$xs = \mathtt{nil})$$
$$\vee$$
$$(\exists hd, q, tl\,.$$
$$p \mapsto (hd, q) *$$
$$\mathtt{IntList}(q, tl) \wedge$$
$$xs = \mathtt{cons}(\mathtt{hd}, \mathtt{tl}))$$

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p)
{
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

Looks executable!

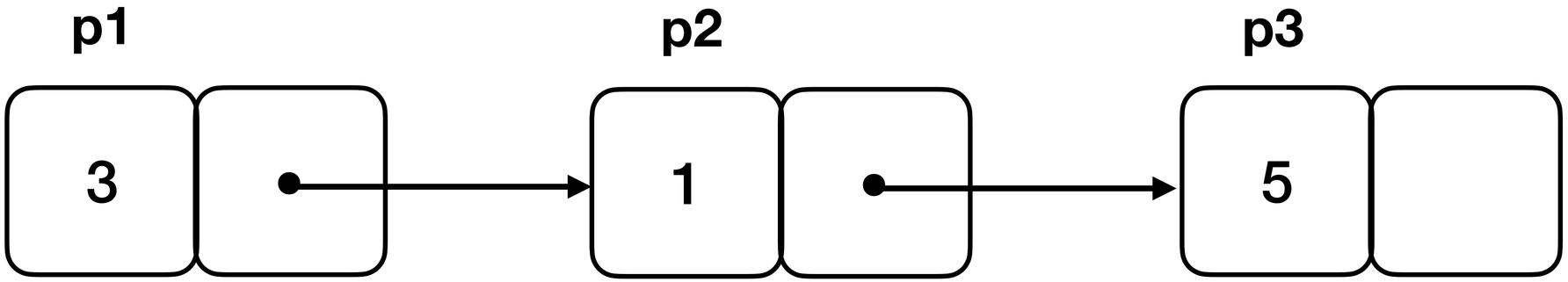
Dynamic ownership checking

- Global ownership ghost state, **tracking ownership per byte**
- Records **function stack depth**
- Extended on allocations, both explicit (calls to **`malloc`**) and implicit
- Checked and modified at pre- and post-conditions

$$\textbf{take}\ v = \texttt{Owned} < ctype > (p)$$

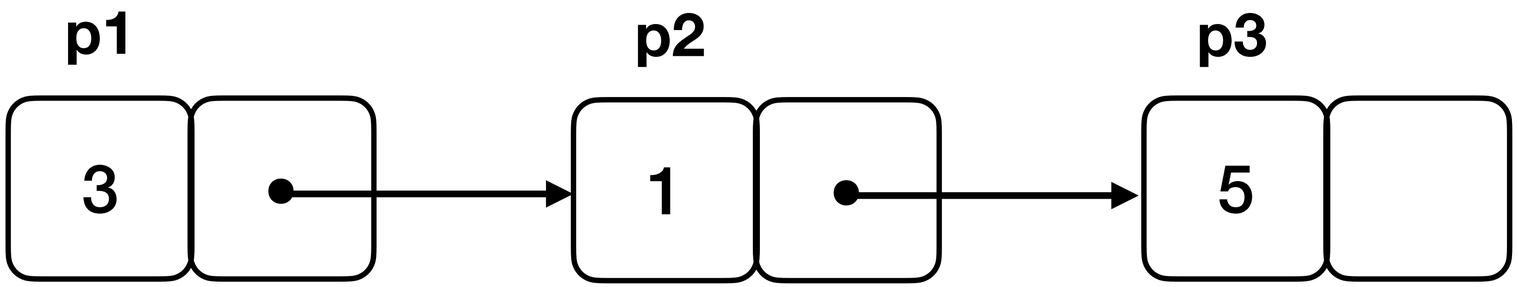
$$
\begin{aligned}
&\texttt{IntList}(p, xs) = \\
&\quad (p = \texttt{NULL} \wedge \\
&\quad\ xs = \texttt{nil}) \\
&\vee \\
&\quad (\exists hd, q, tl\,. \\
&\quad p \mapsto (hd, q) * \\
&\quad \texttt{IntList}(q, tl) \wedge \\
&\quad xs = \texttt{cons}(\texttt{hd}, \texttt{tl}))
\end{aligned}
$$

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```



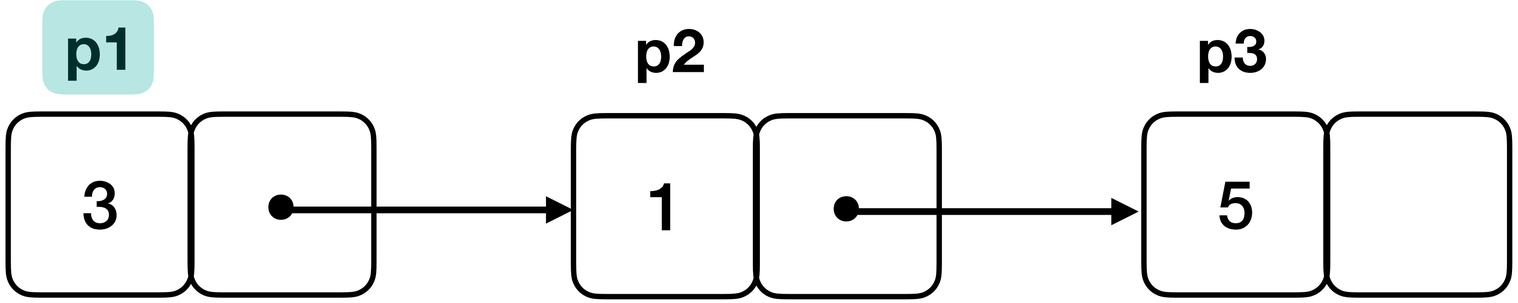
```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```



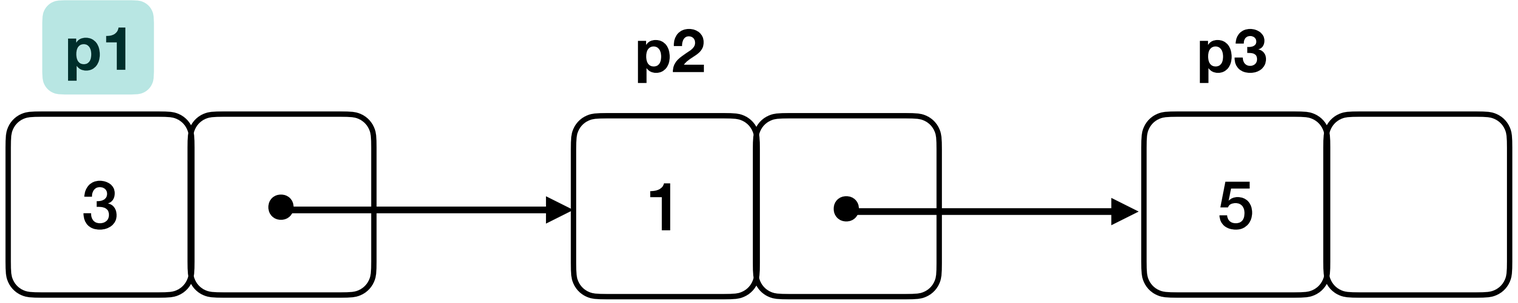
```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```



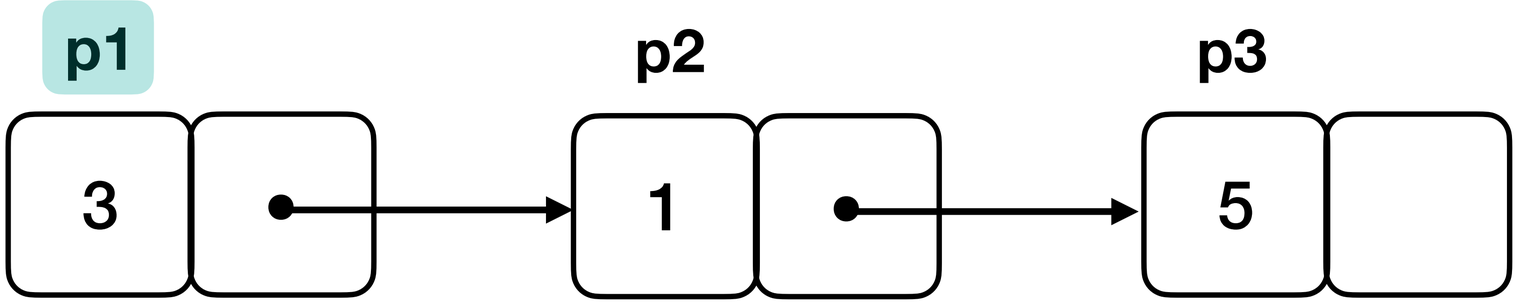
```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```



```
/*@ requires take L1 = IntList(p1); @*/
```

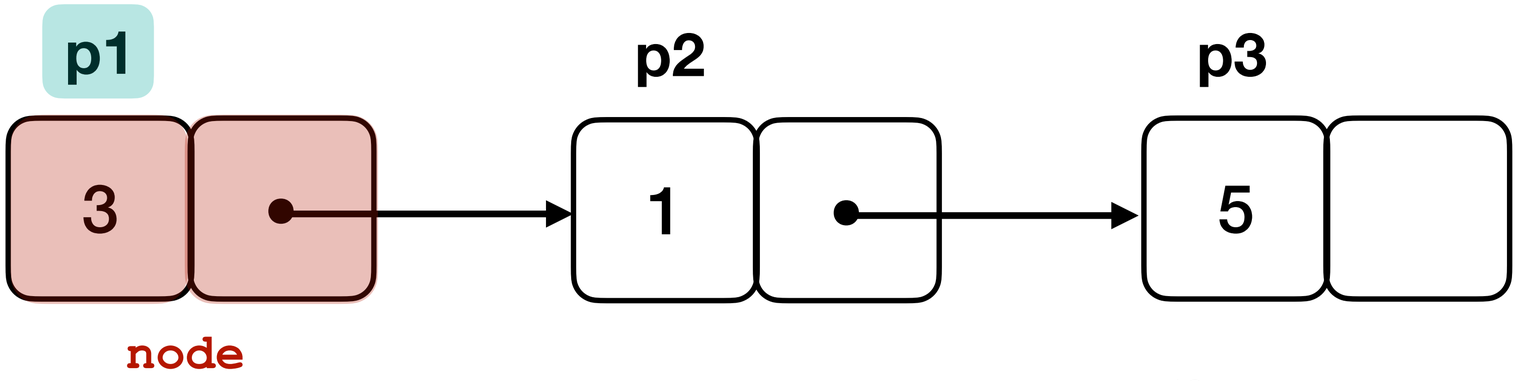
```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

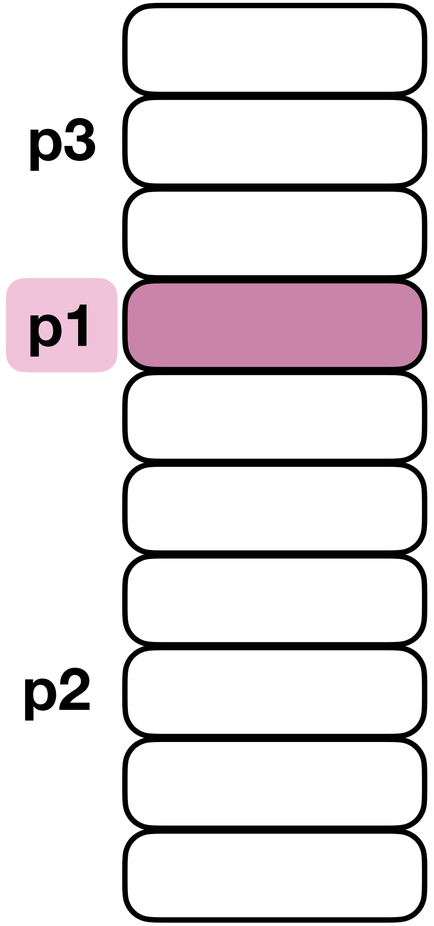


```
struct node
  {int x; struct node *next;};
```

```
/*@ requires take L1 = IntList(p1); @*/
```

```
predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

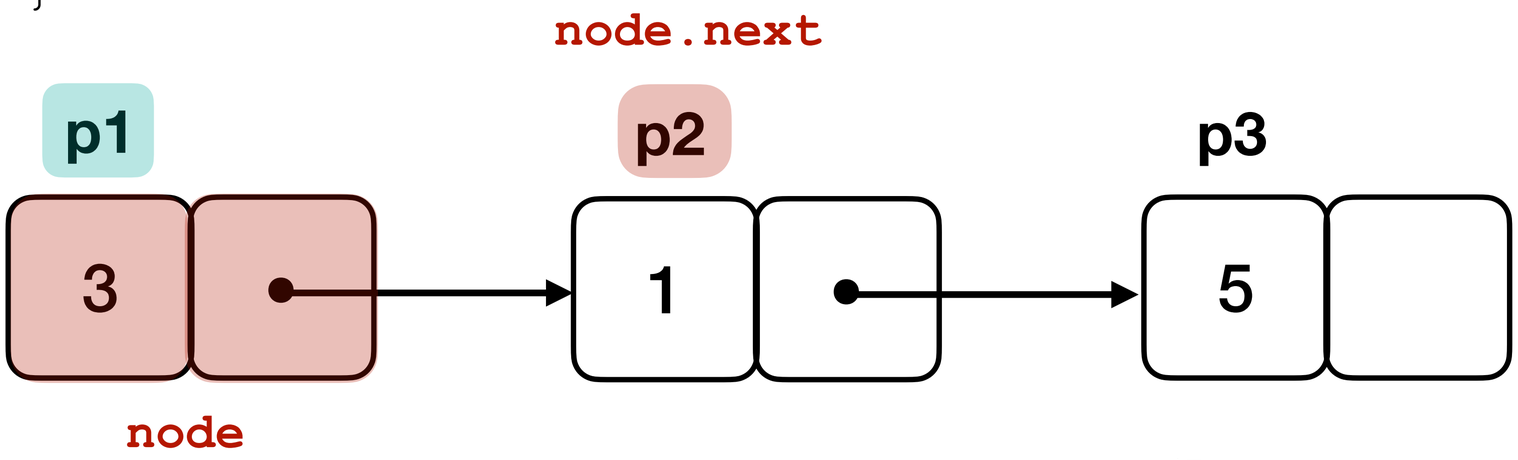


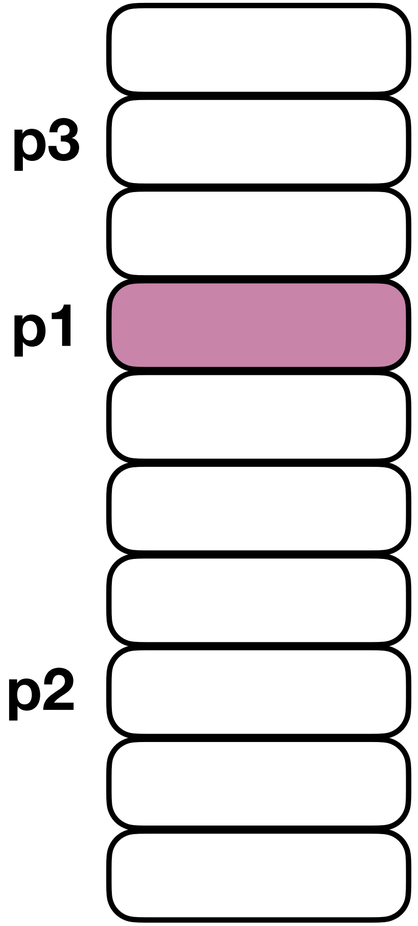


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

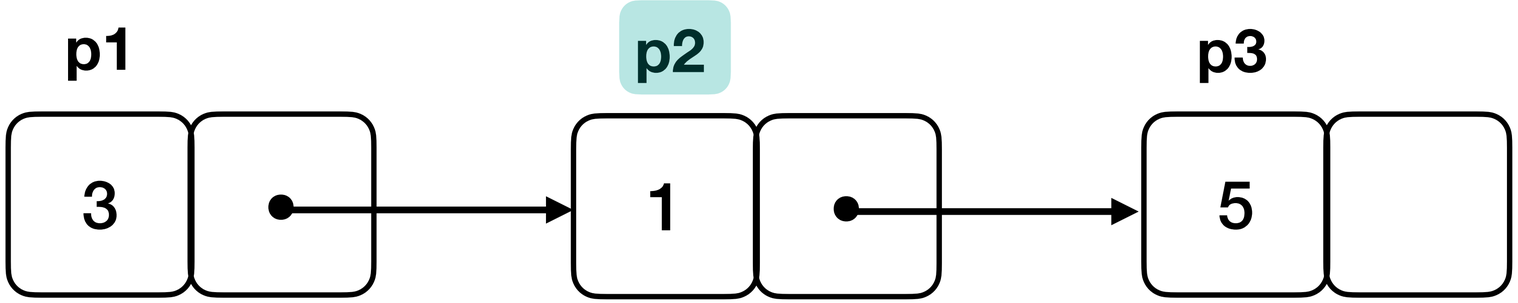


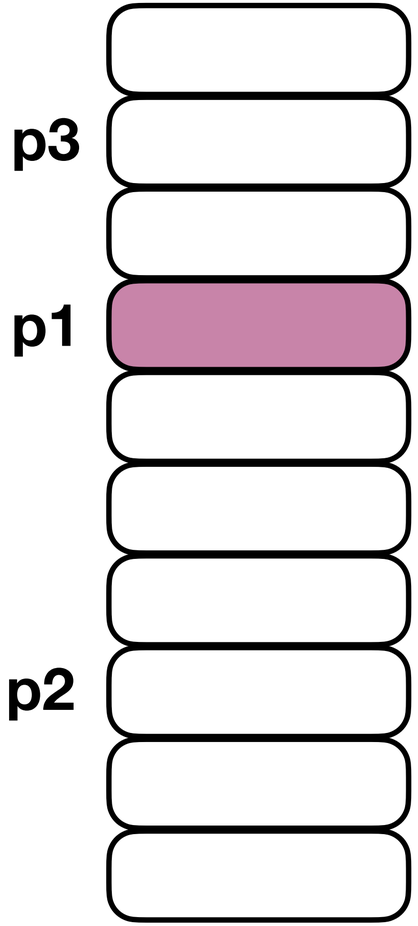


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

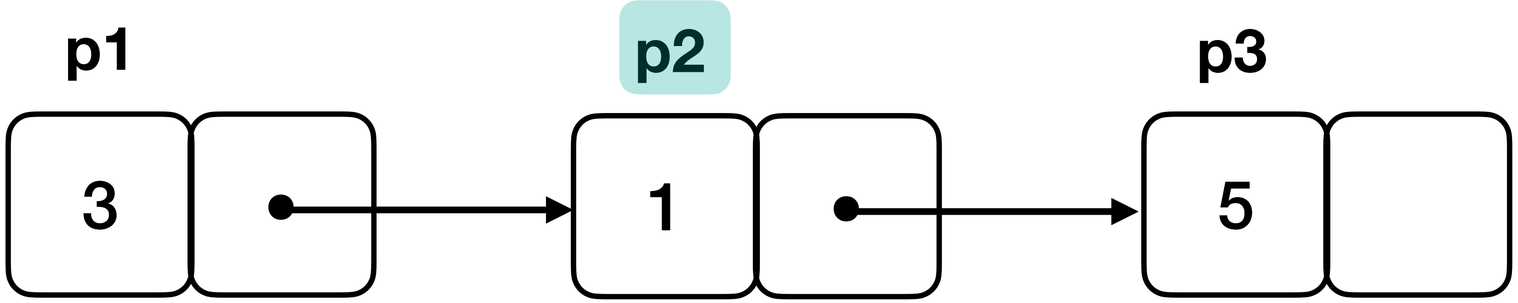


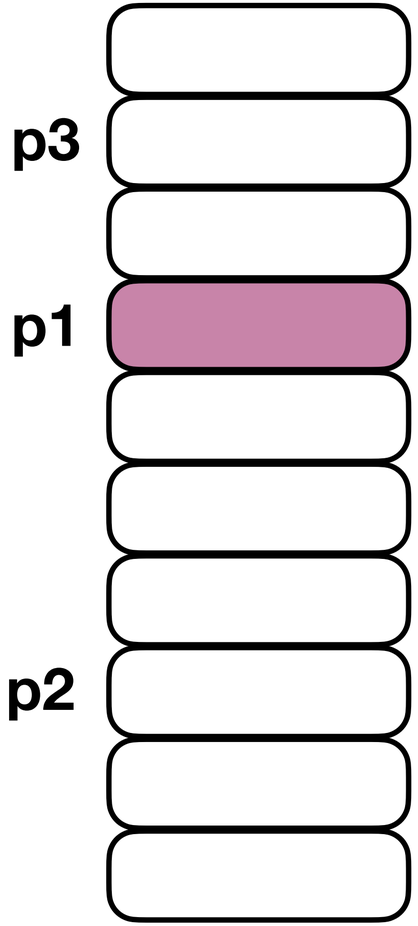


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

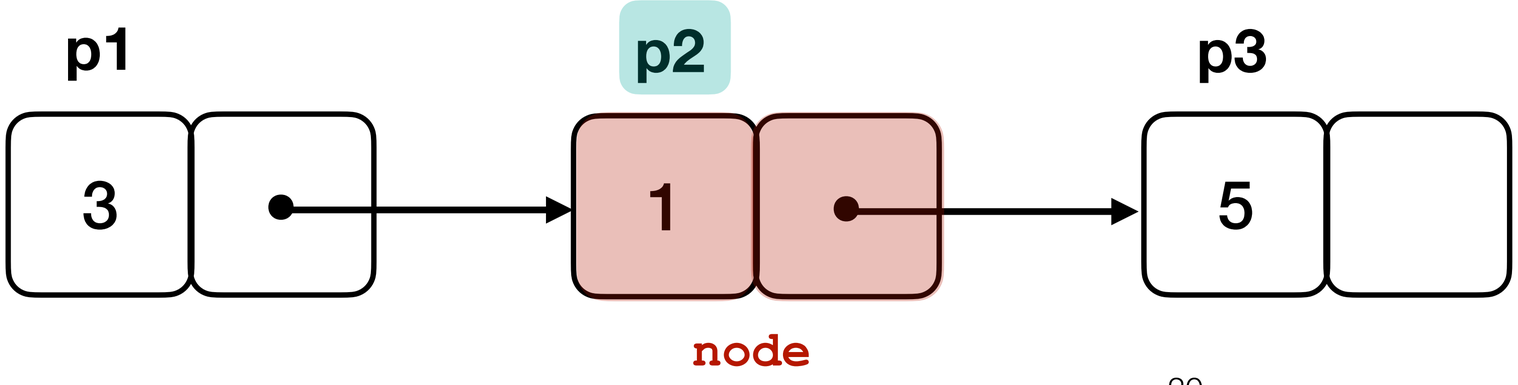


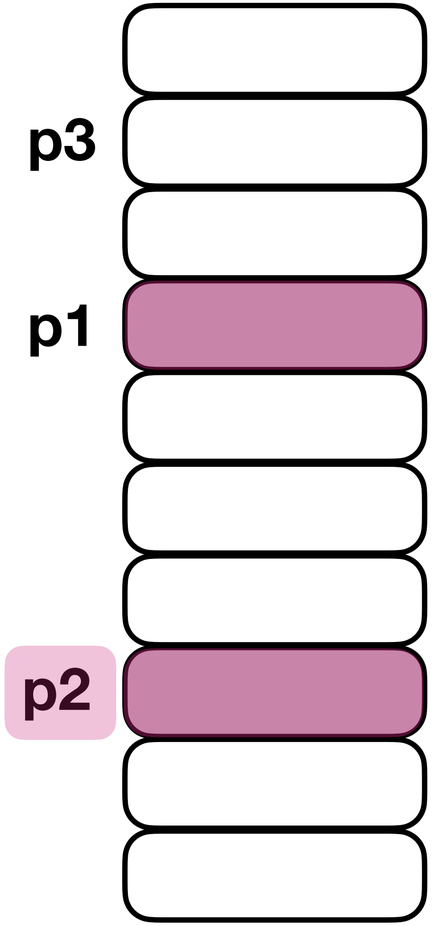


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

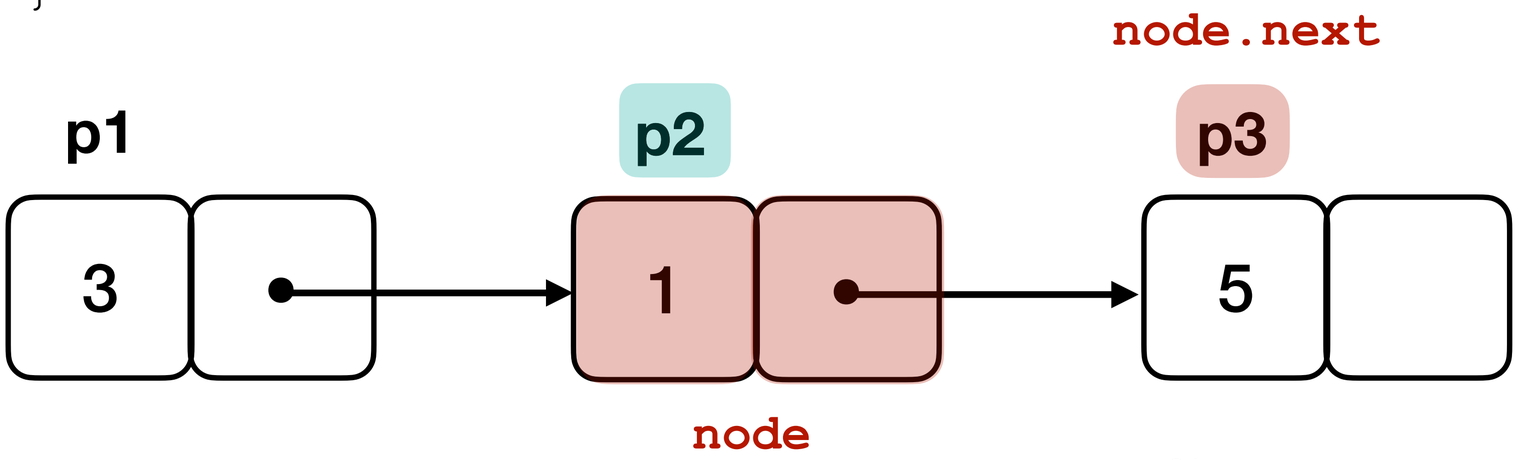


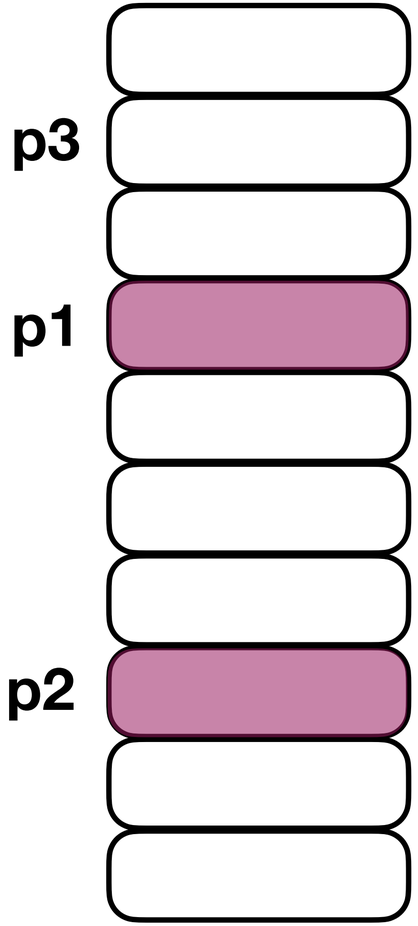


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

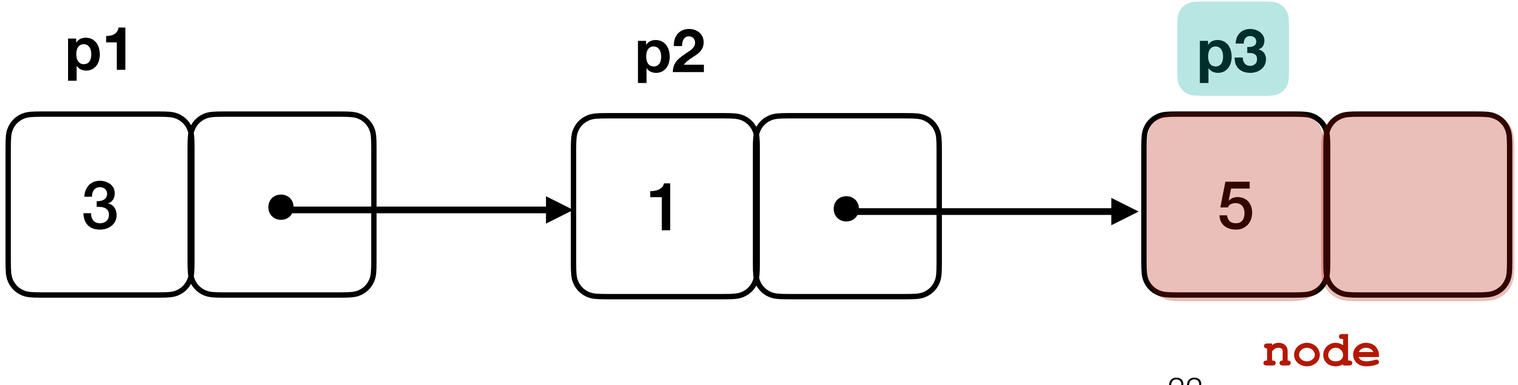


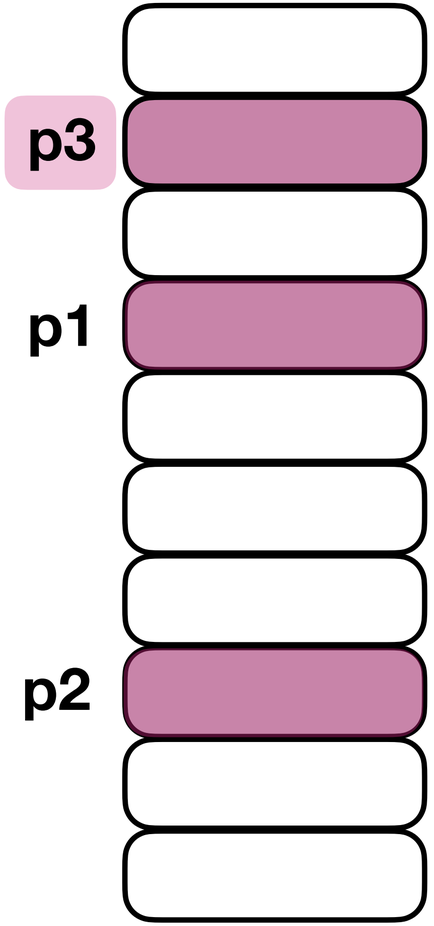


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

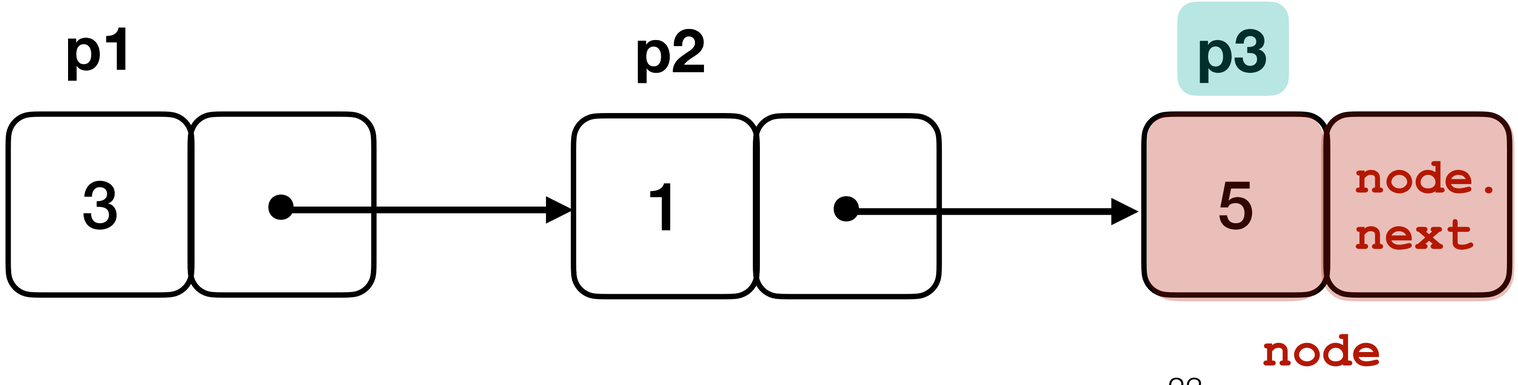


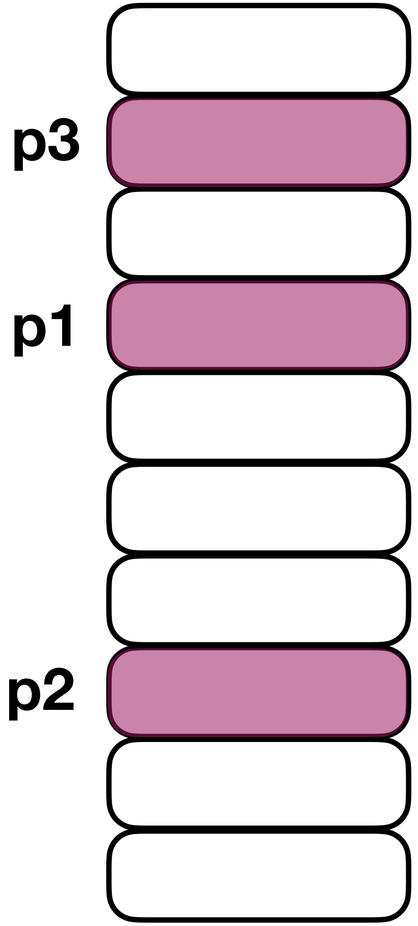


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

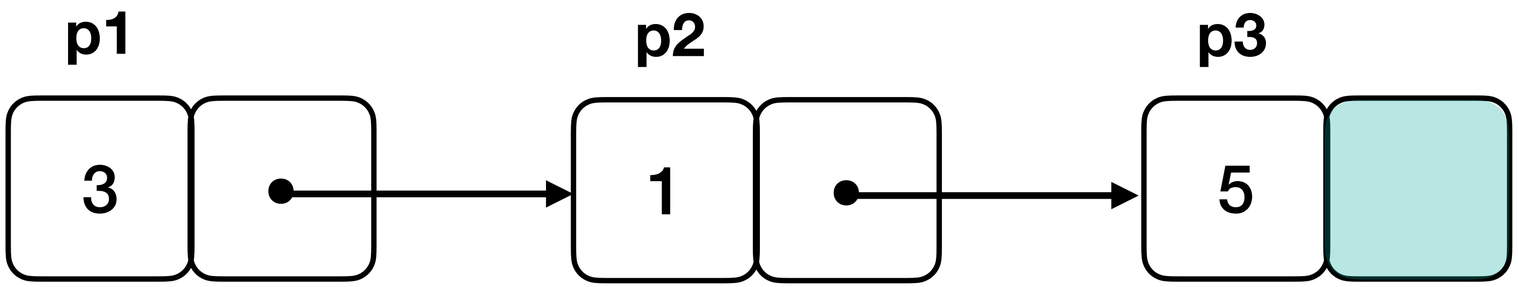


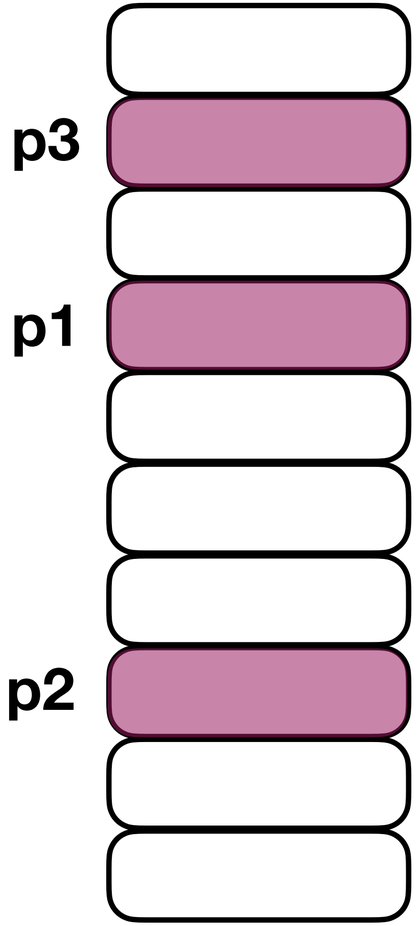


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

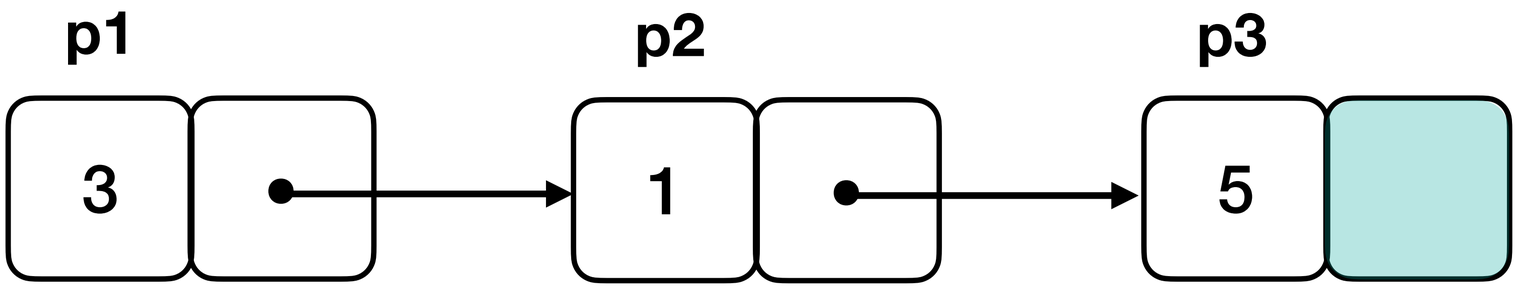


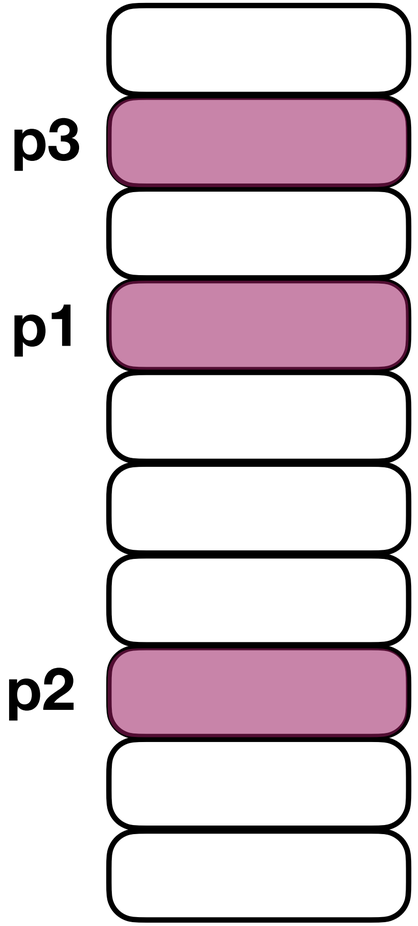


Global ownership ghost state

```
/*@ requires take L1 = IntList(p1); @*/
```

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```



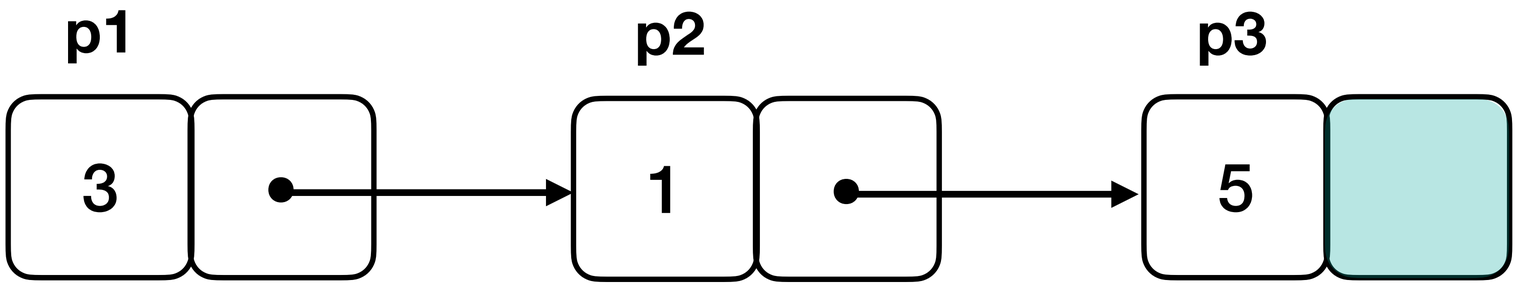


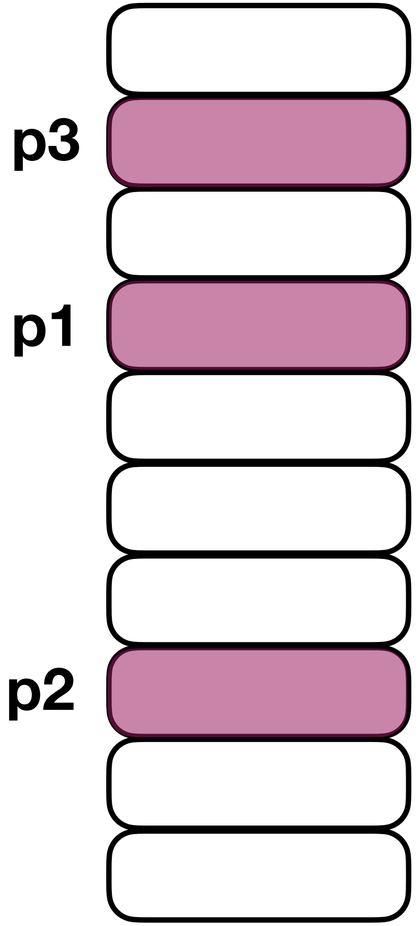
Global ownership ghost state

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

```
/*@ requires take L1 = IntList(p1); @*/
```

Cons {hd: **3**, tl: Cons {hd: **1**, tl: Cons {hd: **5**, tl: Nil {} } } }



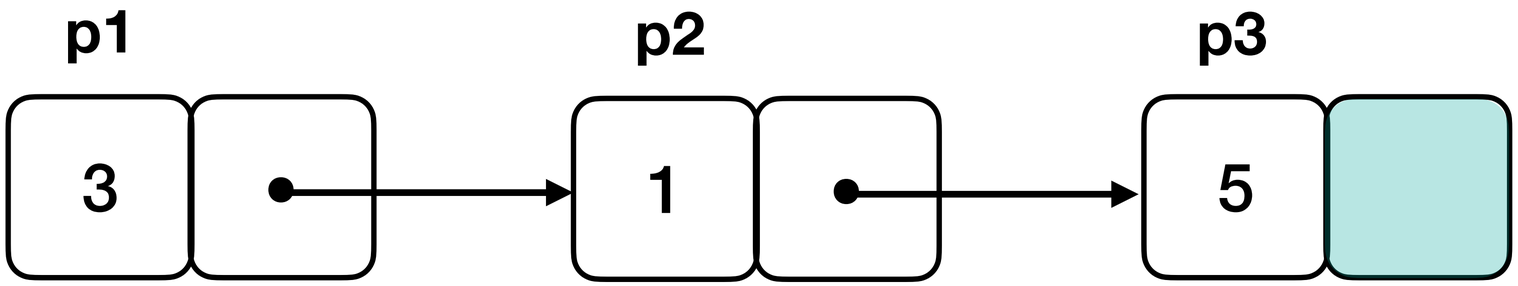


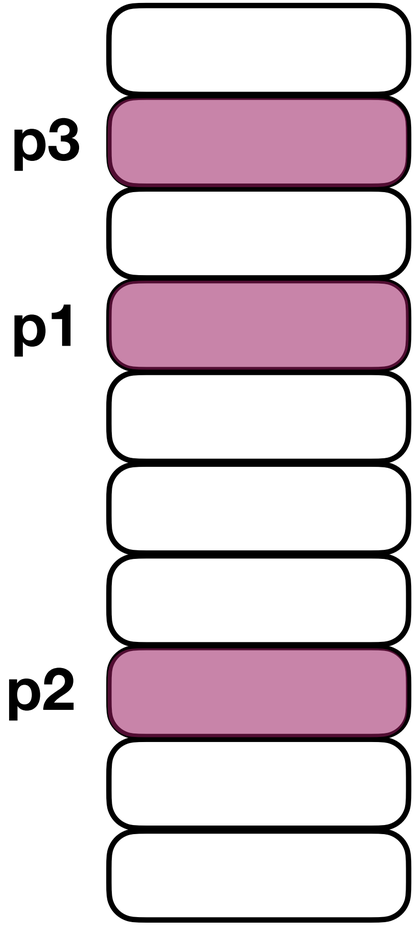
Global ownership ghost state

```
struct node
  {int x; struct node *next;};

predicate integer_list IntList (pointer p) {
  if (p == NULL) {
    return (Nil {});
  }
  else {
    take node = Owned<struct node>(p);
    take tl = IntList(node.next);
    return (Cons {hd: node.x, tl: tl});
  }
}
```

```
/*@ ensures take L2 = IntList(p1); @*/
```

Cons {hd: **3**, tl: Cons {hd: **1**, tl: Cons {hd: **5**, tl: Nil {} } } }





Global ownership ghost state

Dynamic ownership checking

$$\frac{\begin{array}{l} H(n) = n' \\ \text{runtime_check_res}\,(n \in R_2) \quad R_2 = R_2' * \texttt{Owned}\,(n) \quad R_1' = R_1 * \texttt{Owned}\,(n) \\ \text{runtime_check_imp}\,(G(n) = sd) \quad G = G_1 * n \mapsto sd \quad G' = G_1 * n \mapsto incdec(sd) \end{array}}{P, E_g, H, incdec, sd \vdash \langle \texttt{Owned}(n),\ V,\ R_1,\ R_2,\ G\rangle \longrightarrow \langle \texttt{return}\ n',\ \cdot :: V,\ R_1',\ R_2',\ G'\rangle}$$

SPEC_PRED_OWNED

Formalisation + ***proof*** *that runtime ownership ghost state correctly captures explicit resource passing*

Demo!

Application: pKVM

Hypervisor in development at Google to protect Android Linux kernel

- Fulminate applied to CN-annotated *buddy allocator*
  - C source: **320 LoC**, CN specs: **702 LoC**
  - $\approx$ **30 s** to run Fulminate, compile and run binary

(Some) challenges so far

- Dealing with the C
- Making it EL2-friendly (WIP)
- Keeping the instrumented source readable
  - + amenable to use with common developer tools like gdb
- Improving CN without breaking its executability

Further work: CN property-based testing

- Bennet (UPenn): random test generator for exercising Fulminate-produced runtime assertions on concrete inputs
- Every generated test satisfies precondition

Bennet: Randomized Specification Testing for Heap-Manipulating Programs

ZAIN K AAMER, University of Pennsylvania, USA
BENJAMIN C. PIERCE, University of Pennsylvania, USA

Property-based testing (PBT), widely used in functional languages and interactive theorem provers, works by randomly generating many inputs to a system under test. While PBT has also seen some use in low-level languages like C, users in this setting must craft all their own generators by hand, rather than letting the tool synthesize most generators automatically from types or logical specifications. For low-level code with complex memory ownership patterns, writing such generators can waste significant amounts of time.

CN, a specification and verification framework for C, features a streamlined presentation of separation logic that is specially tuned to present only "easy" logical problems to an underlying constraint solver. Prior work on the Fulminate testing framework has shown that CN's streamlined specifications can also be checked effectively at run time, providing an oracle for testing whether a memory state satisfies a pre- or postcondition.